Faculty of Natural, Mathematical, and Engineering Sciences
Department of Informatics

Software Engineering Group Project

Timeout

Galatasaray CS
Bader Al Homood
Elif Haciyanli
Erin Ozdemir
Duru Atay
Juyeop Kim
Hikmet Ozan Kaya
Som Ajmera

March 30, 2026

Contents

1 Introduction **1**
1.1 Summary . . . 1
1.2 Software Used . . . 1

2 Product **2**
2.1 Objectives, context and key stakeholders . . . 2
2.2 Requirements and specifications . . . 2
2.3 Features . . . 4

3 Design **6**
3.1 System Overview . . . 6
3.2 Middleware . . . 7
3.3 Views and Service Layer . . . 7
3.4 Data Model . . . 8
3.5 Authentication . . . 9
3.6 Deployment . . . 9

4 Testing **10**
4.1 Approach to software testing: . . . 10
4.1.1 Reliance on testing: . . . 10
4.1.2 Manual testing: . . . 10
4.2 Limits of testing: . . . 11
4.2.1 The limits of automated testing: . . . 11
4.2.2 The limits of manual testing: . . . 11
4.3 Manual Tests Appendix . . . 12
4.3.1 Follow requests: . . . 12
4.3.2 Buttons: . . . 13
4.3.3 Banning users and flagging system: . . . 14
4.3.4 Focus mode and Pomodoro timer: . . . 15

5 Project management **16**
5.1 Management . . . 16
5.2 What Worked Well . . . 16
5.3 What Didn't Work Well . . . 16
5.4 What The Team Learned . . . 16

Chapter 1

Introduction

1.1 Summary

Modern student life is overwhelming. Lectures, deadlines, part-time work, and social commitments all compete for the same limited hours, the result is that students are left feeling isolated and burdened. Staying connected with friends becomes something that requires effort rather than something that happens naturally. Modern tools that most students rely on, such as mainstream social media or shared calendars, were never designed with this specific tension in mind. Timeout was built to address this directly. At its core, it is a time management application which gives students a single space where their academic and social lives are visible together, making it easier to plan around commitments, identify free time, and coordinate with others without the overhead of juggling multiple apps. For students, the value is straightforward: better visibility over their time leads to less friction between wanting to connect and actually doing it. For universities and student organisations, a student body that can manage its time and social life in one place translates to stronger community cohesion, reduced isolation, and better overall wellbeing outcomes.

1.2 Software Used

Timeout is a web-based application accessible through any modern browser on both desktop and mobile devices. The backend is built on Django, a mature and well-structured Python web framework, which handles routing, authentication, data-modelling, and business logic. User authentication is managed through Django Allauth, providing support for standard email registration as well as Google OAuth for a smoother sign-in experience. The frontend is composed of HTML templates rendered server-side by Django, styled with Bootstrap for responsive layout and expended with custom CSS that implements a visual design such a consistent blue-to-teal gradient colour system. Vanilla JavaScript is used throughout the frontend to handle dynamic interactions, making use of the Fetch API for asynchronous communication with the backend without requiring full page reloads. The database layer uses a relational model compatible with PostgreSQL, managed through Django's ORM. Django's signal framework is used to handle asynchronous side effects, keeping core application logic clean and decoupled.

Chapter 2

Product

2.1 Objectives, context and key stakeholders

University students face a growing challenge in managing the competing demands of academic life. Deadlines, examinations, lectures, and group work must all be balanced alongside personal wellbeing and social engagement. Existing tools for planning, note-taking, and communication are often spread across different platforms, increasing cognitive load and making it harder to maintain a consistent workflow. Timeout was developed in response to this fragmentation by bringing these activities together in one student-centred environment.

Key Stakeholders

The primary stakeholders are **university students**, who benefit directly from improved organisation, stronger focus, and better awareness of academic commitments. Secondary stakeholders include **teaching staff and institutions**, who benefit indirectly when students are better equipped to meet deadlines and engage with coursework.

Objectives

1. **Reduce missed deadlines**: Provide students with a centralised view of all upcoming academic obligations so that nothing falls through the cracks.
2. **Encourage consistent study habits**: Give students structured tools and incentives to build and sustain regular study routines.
3. **Maintain social connection during periods of intensive study**: Allow students to stay engaged with their peers without leaving the productivity environment.
4. **Improve accessibility and personal comfort**: Ensure the application is usable for students with varying visual needs and interface preferences.

2.2 Requirements and specifications

Based on these objectives, the team identified a set of core requirements for the delivered system. They are expressed as user stories and grouped by the objective they serve.

In addition to the functional requirements below, the team identified several non-functional requirements: the application should be responsive across desktop and mobile browsers, accessible to users with common colour vision deficiencies, and deployable as a Progressive Web App (PWA) to improve installability and mobile usability.

Objective 1: Reduce missed deadlines

- *As a student, I want to view all my deadlines and events in a single calendar so that I can plan my time effectively.* This is supported by the calendar and deadlines

views at `/calendar/` and `/deadlines/`.

- *As a student, I want to create calendar events using natural language so that scheduling is faster.* Available through the AI-assisted event creation feature at `/ai-calendar/`, powered by the OpenAI API.
- *As a student, I want to receive notifications for upcoming deadlines so that I am reminded in advance.* Implemented via the notification system at `/notifications/`.

Objective 2: Encourage consistent study habits

- *As a student, I want to use a Pomodoro timer to structure my study sessions into focused intervals so that I can maintain concentration.* Implemented in the notes editor and accessible from the dashboard.
- *As a student, I want to earn experience points and level up as I study so that I feel motivated to continue.* XP is awarded for completing focus sessions, creating notes, and maintaining daily streaks, with level progress displayed on the dashboard.
- *As a student, I want to take and organise notes so that I can manage my study materials effectively.* Provided through the notes interface at `/notes/`, with support for categories, pinning, and due dates.
- *As a student, I want to track my study activity over time so that I can identify patterns and improve.* Available on the statistics page at `/statistics/`, featuring daily focus charts and a study activity heatmap.

Objective 3: Maintain social connection

- *As a student, I want to share posts and updates with my peers so that I can stay connected while studying.* Provided through the social feed at `/feed/`.
- *As a student, I want to send and receive private messages so that I can communicate with classmates directly.* Implemented via the messaging interface at `/inbox/`.
- *As a student, I want to follow other users and see their activity in a personalised feed so that I can discover relevant content.* Supported by the Following tab in the social feed.

Objective 4: Improve accessibility and personal comfort

- *As a student with a colour vision deficiency, I want to switch to a colourblind-friendly colour scheme so that I can use the application comfortably.* Implemented in settings with Protanopia, Deuteranopia, and Tritanopia modes.
- *As a student, I want to adjust the font size and toggle dark mode so that the interface suits my preferences.* Available through the settings page, with font size adjustable between 80% and 150% and light, dark, and system-default theme options.

Objective 5: Regulate and maintain a safe environment

- *As an admin, I want to be able to view and edit/delete harmful content so that users/students can continue using the app while having a safe environment provided*

for them. Available from the admin model, able to review flagged content and block users based on any harmful activity.

- *As an admin I want to review flagged content and take the necessary precautions.* The admin can log in to review any flagged content such as posts, comments or messages and ban users or posts if they are behaving in a manner that endangers the well-being and online safety of other users.

2.3 Features

Calendar

The calendar feature can be found on the navbar next to the dashboard. Provides a monthly calendar with weekly grid view. Users can track and add new events either by clicking on the dates or on the add event button. If not specified, the event defaults from 9.00 am to 10.00 am for the given date. Users can choose to add different types (Deadlines, Classes, Exams, etc), time and date, place, and any additional information to the event. They can choose to make the event public or private and recurring, which displays the event throughout the days it lasts. The calendar also shows global events such as national holidays or important dates.

AI Features

On the calendar page users can access AI planning features through the AI button on top of the grid. The function consists of three features for users to choose from: Plan Study, Reschedule Sessions, AI Add Event. Plan Study feature allows users to pick an event and establish the hours they want to spend studying so the OpenAI model can find unused slots and assign a balanced study session leading up to the event. Reschedule Sessions feature works for the cases where a user is unsatisfied with their current workload so they have the option to re-plan their study hours based on their remaining empty slots. AI Add Event feature is an alternative way to add an event without using the normal add event button.

Deadline List

There is a toggle button on the calendar that switches the calendar from the monthly grid to a list view of the deadlines. The events are listed and can be marked as complete/incomplete, filtered by event types, sorted through urgency. Each event shows if the user is on track, past due or upcoming and provides how much time has left until deadline. Users can also choose to add events on the deadline list view without having to go back to the calendar.

Social

User can access the feature from the navbar. They can create, edit, delete, like posts, write comments. Creating a public event creates a post. The posts can be flagged for harmful content and be reviewed by admin to delete or dismiss. Users can also message each other. This feature can be found in the social page.

Profile/Setting

The user can modify their profile by clicking their profile picture on the top right and clicking the profile button. They can view their followers/following, posts and activity status. Through the settings button on the profile page or under the drop-down view from the profile picture users can modify their settings. They can choose to make their page public/private, add a profile picture, modify their username or password, choose colour modes for dark vision or colour blindness. The user can also choose to delete their profile resulting in the erasure of their content.

Statistics

User can access the feature from the navbar. They can view their statistics and progress to track their productivity and time management. They can see how much time they have spent in focus mode as well as view the top users for the week. The page presents a pie chart, bar and plot graphs for activity tracking and gives out an sum of total events at the top.

Dashboard

The dashboard is the default page that is accessed when the user is first logged in the app. It provides the user with a AI generated weekly briefing or their progress, enables the user to see a quick review of their stats and any upcoming deadlines and events as well as giving quick links to other features such as latest notes, messages and social feed.

Following

Users can follow other users through their profiles. If a profile is private the user needs to send a follow request that will be accepted by the other user. This is to protect the privacy of the students/users that are benefiting from the app.

Note/Focus

In the notes page users can choose to create a note. The note taking function has the option to view as pages or pageless and font/size edits for the texts. The notes page also includes timer and a focus mode which disables users from switching between pages to create a more productive environment.

Notifications

The user can access their notifications about upcoming events, posts, messages on the notifications page found in the navbar to stay on top of their deadlines and messages. They can mark the messages as read or unread and delete when they are done reviewing them. The notification count is present at the top right side of the notification button on the navbar and it corrects to 99+ when the notification count exceeds the threshold to avoid overwhelming the user with a high number of alerts.

Chapter 3

Design

3.1 System Overview

Timeout is a time management web application built with Django 5.0 and Python, targeting university students who need to manage their academic and social lives in one place. The application integrates a social feed, calendar and event management, note-taking with a Pomodoro timer, direct messaging, and an AI-powered dashboard briefing into a single platform.

As shown in Figure 3.1, the system follows a layered architecture. HTTPS requests from the browser are received by an Nginx reverse proxy on an EC2 server, which forwards them to the Django application. Within Django, requests pass through a Views layer, then a Services layer, before reaching the Models layer which reads from and writes to a SQLite database. The application also communicates with three external services: Google OAuth for social login, the OpenAI API (`GPT-4o-mini`) for AI features, and SendGrid for transactional email.

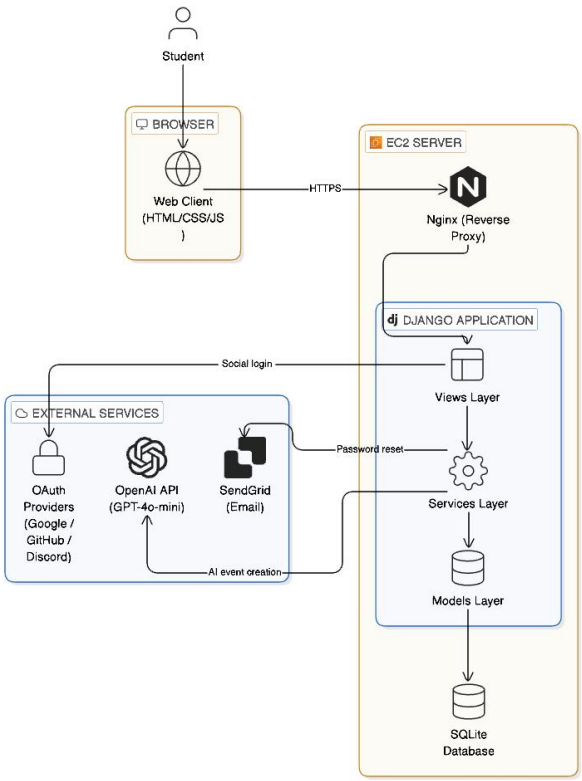

Figure 3.1: Component Diagram

3.2 Middleware

Django processes every HTTP request through a stack of middleware before it reaches a view, and Timeout configures nine layers in `settings.py`. The standard Django middleware handles security headers (`SecurityMiddleware`), session management (`SessionMiddleware`), CSRF protection (`CsrfViewMiddleware`), and clickjacking prevention (`XFrameOptionsMiddleware`). The `AuthenticationMiddleware` resolves the logged-in user from the session, and django-allauth's `AccountMiddleware` supports social authentication flows. Django's `MessageMiddleware` and `CommonMiddleware` round out the stack.

The one custom middleware is `BannedUserMiddleware`, defined in `timeout/middleware.py`. It sits immediately after `AuthenticationMiddleware` so that the current user is already resolved when it runs. On each request it checks whether the authenticated user has the `is_banned` flag set; if so, it logs them out and issues a redirect to the `/banned/` route. The `/banned/` path itself is exempted from this check to prevent an infinite redirect loop. All other middleware in the stack is standard Django or provided by django-allauth.

3.3 Views and Service Layer

Timeout separates request handling from business logic across two distinct layers. The URL configuration is split across per-feature modules covering authentication, pages, social, calendar, messaging, notes, notifications, and study planning, each mapping to a corresponding views file.

The views layer handles incoming HTTP requests and returns responses. Authentication views manage signup, login, logout, and profile completion. The dashboard view assembles the main page by gathering upcoming events, notes, deadlines, and an AI briefing. Social views handle post creation and deletion, likes, bookmarks, comments, follow requests, blocking, and user search. Moderation views allow staff to review flagged posts and ban users. Calendar, notes, and messaging each have their own views handling CRUD operations and any associated logic such as conflict detection for calendar events and read status tracking for messages.

Business logic is extracted into a `services/` package to keep views thin. `FeedService` constructs the social feed by querying posts from followed users or public posts for the discover tab, applying privacy and block filters at the query level. `SocialService` handles follow request state, block status checks, and profile visibility. `NoteService` manages note retrieval, search, and category filtering, and also handles gamification by awarding XP on note creation and edits and logging Pomodoro sessions to a daily `StudyLog`. `DeadlineService` retrieves and filters events by urgency and completion status. `NotificationService` creates time-based alerts for upcoming deadlines and events, as well as notifications for social interactions such as likes and messages. `AIService` generates a weekly dashboard briefing by collecting study statistics and calling the OpenAI API, with the result cached per user for 24 hours. `EmailService` sends transactional emails via SendGrid, used for password reset codes. `StudyPlannerService` identifies free time slots in a user's calendar and suggests evenly spaced study sessions across a given period.

3.4 Data Model

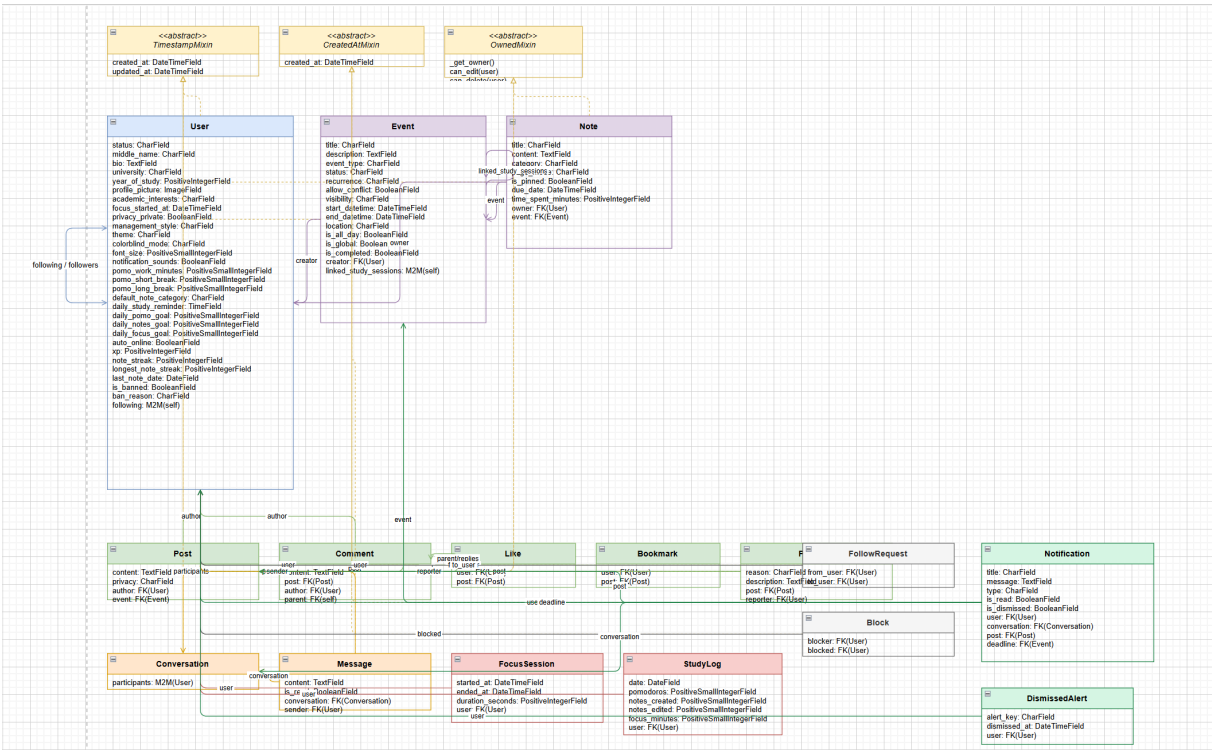

Figure 3.2: UML Class Diagram

The data model is illustrated in Figure 3.2. Three abstract mixins provide shared behaviour across models: `TimestampMixin` adds `created_at` and `updated_at` fields, `CreatedAtMixin` adds `created_at` alone, and `OwnedMixin` provides `can_edit` and `can_delete` methods for ownership checks.

The central model is `User`, which extends Django's `AbstractUser`. Beyond standard authentication fields it stores profile information such as bio, university, and year of study, a self-referential many-to-many `following` relationship that forms the social graph, status and privacy settings, accessibility preferences including theme and colorblind mode, Pomodoro configuration, daily study goals, gamification state via `xp` and `note_streak` fields, and moderation fields `is_banned` and `ban_reason`.

The `Event` model represents calendar entries. Each event belongs to a `creator` (FK to User) and carries fields for type, status, priority, recurrence, visibility, start and end datetimes, location, and conflict handling. A `linked_study_sessions` many-to-many field on `Event` allows study sessions to be associated with deadlines. When an event's visibility is set to public it automatically creates a corresponding `Post`, and the post is removed if visibility is changed to private.

The `Note` model belongs to an `owner` and optionally links to an `Event`. It stores a category, pin state, optional due date, and a `time_spent_minutes` field populated by the Pomodoro timer.

The social layer is built from `Post`, `Comment`, `Like`, and `Bookmark`. Posts have a `privacy` field and a foreign key to both `User` and optionally `Event`. Comments are self-referential via a `parent` field to support replies. Likes and bookmarks each carry a unique constraint

on the user and post pair to prevent duplicates. The `FollowRequest` model manages the pending follow flow for private profiles, with a unique constraint on the sender and recipient pair. The `Block` model records one-directional blocks between users, also with a unique constraint.

Direct messaging uses two models: `Conversation` holds a many-to-many relationship with participants, and `Message` stores the content, sender, read status, and a foreign key back to the conversation.

The remaining models support auxiliary features. `Notification` records alerts of various types (deadline, event, message, like, comment, bookmark, follow) with optional foreign keys to the relevant post, conversation, or event. `FocusSession` logs the start time, end time, and duration of each Pomodoro session. `StudyLog` records a daily aggregate per user of Pomodoros completed, notes created and edited, and focus minutes, used to drive the activity heatmap. `DismissedAlert` tracks which dismissible alerts a user has already seen to prevent them from reappearing.

3.5 Authentication

Timeout supports two authentication paths: local email and password registration, and social login via Google, GitHub, and Discord through django-allauth. Both flows redirect new users to a profile completion page where they provide a username, name, year of study, and university before accessing the application.

For local registration, passwords are validated for strength and checked for similarity against the user's email and username. Login accepts either an email address or a username. Password reset is handled by emailing a six-digit code via SendGrid, which expires after ten minutes.

A custom `TimeoutSocialAccountAdapter` handles the `pre_social_login` hook to automatically link an incoming social account to an existing local account if the email address matches, preventing duplicate accounts. Passwords are hashed using Argon2 as the primary hasher, all authenticated views are protected with Django's `login_required` decorator, and staff-only moderation actions are gated on the `is_staff` flag.

3.6 Deployment

Timeout is hosted on a Linux server and deployed on an automated Github-based deployment workflow. Whenever a change gets pushed to the main branch, the server pulls the latest codebase. Then it installs any new dependencies from `requirements.txt`, runs database migrations, and collects static files, before restarting the application service. Static assets like CSS, JavaScript, and images are handled by the web server, while the Django application runs behind Nginx via Gunicorn . All communication with external services such as OpenAI, SendGrid, and OAuth providers are handled by HTTPS. Currently, the system uses a single SQLite3 database file stored locally on the server alongside the application.

Chapter 4

Testing

4.1 Approach to software testing:

The team approached software testing through a variety of extensive automated and manual testing, as well as ensuring test quality, DRY / WETNESS, and code coverage to meet all marking requirements.

4.1.1 Reliance on testing:

Automated:

Majority of our testing relied on the automated testing via the test files, and together, the team have written 1183 tests for the project, with a code coverage of 97%. During the development of these test files, the team ensured branch coverage was also considered, and every untested line was checked using the coverage report feature in the terminal. The aim was to ensure the test files covered near or at 100% in each file.

The team also used automated testing to check all marking requirements were met for code quality, running scripts often to find lacking areas. These tests included whether there were any files over 400 lines, any functions over 25 lines, missing comments and headers, unused imports, functions, or CSS, as well as inline JavaScript in HTML. All files use the same styling for function names, indentation, and written code.

Manual:

A large portion of our testing also involved manual methods, more specifically for front-end features. As the automated tests cover all functions and features from the back-end and database aspect of the project, the manual testing was to verify that visual and front-end behaviours worked as intended.

4.1.2 Manual testing:

Purpose of manual testing:

The purpose of manual testing in our project was to find any front-end bugs and errors that could not be automatically tested via the test files. The actions taken by the tester included going through each page and manually clicking on all the buttons as well as checking visually the CSS displayed as intended.

Extent of manual testing:

The team relied on manual testing for many aspects of the project that would have ideally been automated using external tools, however, the team opted to take on the front-end testing manually to include visual and interactive features, and chose to use automated testing via the test files for all back-end and database logic.

4.2 Limits of testing:

4.2.1 The limits of automated testing:

Automated testing is an exceptional method of testing the project, however, there are various limitations, as there are numerous aspects that cannot be tested automatically using the written test files.

Front-end testing:

One of the key aspects of the project that cannot be tested automatically is the front-end. The test files are unable to determine if the HTML and CSS are working as expected and intended by the group. This includes whether the buttons redirect the user to the correct link or page, and if the expected pop-ups and warnings, notifications, and reminders appear visually on the app when necessary.

CSS and HTML cohesion:

Another feature to ensure a smooth experience for a user when using the app is to keep the 'look' of the app consistent throughout. It is important to have a clean look, with the same colour scheme and formatting for features, buttons and links on each page. Since the automated tester cannot determine whether this has been kept up, it is an important job of the tester to go through every possible page, link, button, and any other features to keep this cohesion the same throughout.

HTML Buttons:

HTML files can often have bugs that are easy to miss if there is no tester checking every button and link on the page. The automated testing can test and determine whether the actual functions and code work, however it is impossible to determine if the user will have the same experience with the features without manually testing the buttons themselves. There might be errors with the HTML URL links that forbid the user from accessing the feature they are trying to use, i.e. a delete button that is linked incorrectly and does not work, even if the code itself is perfect. This causes the button to never call the delete code, and will be left with an empty and useless feature on the page that is sure to create confusion.

4.2.2 The limits of manual testing:

Manual testing is a crucial aspect of software testing as it covers every test that can not be automated. Ideally, the tester would be able to test every feature to perfection, however, manual testing raises the risk of human error.

Easy to miss a bug:

With continual development and testing, it is unfortunately very easy to miss a bug, which can have different levels of impact on the project that can be difficult to identify. These can be caused by natural and unavoidable human error, as well as the tester experiencing fatigue. A missed bug in manual testing can have both severe and minimal impacts on

the project, which is why it is a crucial job to thoroughly test everything to ensure that the user experiences the intended project cleanly.

Time consuming:

With bigger projects, the code, pages, and features grow exponentially and can make it increasingly difficult to manually test every feature that the app provides. Although it is necessary, it can slow the development process of the app and can be extremely time consuming. The tests are also hard to repeat frequently if changes have been made since the last test. An example of this would be when the tester identifies a bug, and once the developers push new code that fixes said bug, it may have broken another aspect of the project that the tester will have to revisit and discover.

Bias:

As the teams are not made up of many people, the developers of the code are also the testers in our project. This can cause tester bias when checking all features work as intended. Since the tester wrote the code themselves, there is a level of confidence in the code that an unbiased tester would not have, and they may easily overlook a feature that does not work well. This works the same on the contrary, where a tester who does not know the code well may miss a feature that does not appear and would never know that there is a bug or code that is not accessed.

4.3 Manual Tests Appendix

4.3.1 Follow requests:

Step-by-step Action:

A single tester can test this on the local host, or two testers can test this on the deployed version to reduce time spent.

1. Create two separate accounts, where at least one is a private account.
2. Have one account send a follow request to the private account from their profile page
3. Log in to the private account and check notifications or the profile page for the follow request
4. Accept the request and repeat test with a private account that will reject the request

What is expected to be observed?

- The follow button should visually change to determine that a request has been sent and display to the user sending the request that their request has gone through.
- The user receiving the request should have a notification in their notifications page which should display the username of the account requesting to follow them. The buttons on this notification should say Accept and Reject, instead of the Read and Delete that can be found on other notifications. Only once the user has clicked either

button, the notification should be marked as read, greyed out, and the standard buttons should appear.

- The tester should also observe the profile page and see a section with follow requests and the correct user requesting to follow them, which they can choose to follow or reject from there.
- The following / follower count on the profile page should display the correct amount, and when clicked on, the user should be able to see the correct username of the account that has followed them. When checking followers, a follow back button is expected to be seen for users that the account does not follow.
- If the user follows the account back, the tester should observe a count increase in the friends section, and when clicked, they should only be able to see a list of accounts that both follow them, and that they follow back.

Date Last Performed

This test was last performed on Saturday 28th March 2026

Pass or Fail

When initially tested, this test did not pass, and the team had to identify the bugs in the code and re-test once these had been fixed. The most recent test passed, and the final version of the code works exactly as intended.

4.3.2 Buttons:

Step-by-step Action:

A single tester can test this on either the local host or deployed version of the app, however it is recommended that more than one person takes on the testing for this feature.

1. Attempt to create an account on the app, test every button is clickable, and does what is intended. Ensure to fill in all details as required when signing up.
2. Once in the app, log out and log back in to ensure those buttons work too, testing what happens when an incorrect password, or incorrectly capitalised username / email is entered when logging in.
3. Once logged in, the tester must check the profile page and confirm that the details they entered when initially signing up are all visible and correct.
4. The tester should click the edit profile button, change details, and save again to test if those buttons work.
5. Starting from the dashboard, manual tests and clicks of every button is expected, so that the tester can observe if they are necessary and functional.

What is expected to be observed?

- When Signing Up / Logging In, the buttons should redirect the user to the dashboard, and should visually raise errors if there are incorrect or missing details

- The profile page should display the correct details entered by the tester when signing up. This includes their University, current year, and their username.
- When 'Edit Profile' is clicked, the tester should be redirected to the correct page and be able to click all buttons to change their details. Once this page has been exited, the details should now display the updated version on the profile page.
- The dashboard, statistics, social, notes, notifications, and calendar pages should all be accessible via the navigation bar.
- Clicking links should take the tester to the intended page, i.e., '+ New Note' should redirect the tester to the notes page and automatically open the New Note pop-up

Date Last Performed:

This test was last performed on Saturday 28th March 2026

Pass or Fail:

When initially tested, this test did not pass as a few buttons were linked incorrectly in the HTML files, and the team had to identify the bugs in the code and re-test once these had been fixed. The most recent test passed, and the final version of the code works exactly as intended.

4.3.3 Banning users and flagging system:

Step-by-step Action:

A single tester can test this on the local host, or two testers can test this on the deployed version to reduce time spent.

1. Create a new account and comment on a post under the Discover tab on the Social Feed.
2. Flag a post to let the superuser know that this post has been flagged
3. Log In as John Doe using the details johndoe or john.doe@email.com and password as Password123, and check for flagged posts, use the delete button to delete them
4. Click on a user profile and click the Ban button to ban that user temporarily, then attempt to log back in as the banned user
5. John Doe should now unban the banned user and have the user attempt to log in once the ban has been lifted

What is expected to be observed?

- There should be a flag icon that changes visually once the user flags a post.
- Only user johndoe should see the ban button when a profile is clicked, and this should not be visible to any other user.
- User johndoe should also be the only account to visually see flagged posts and should have appropriate buttons to ignore or delete the posts.

- Once a user has been banned, the button should automatically change to 'Unban' in place of the previous 'Ban' button.
- A user should not be able to log in whilst banned, and there should be a visual display of this on the Log In page.

Date Last Performed:

This test was last performed on Saturday 28th March 2026

Pass or Fail:

When tested, this test passed, and the final version of the code works exactly as intended.

4.3.4 Focus mode and Pomodoro timer:

Step-by-step Action:

A single tester can test this on the local host or deployed version of the app.

1. Enter focus mode via the Notes Page
2. Attempt to click out of the page onto another page in the app
3. Start the Pomodoro timer, pause, restart, skip
4. Add a new note or edit existing notes, and check if the page remains in focus mode

What is expected to be observed?

- There should be a red pop-up at the top of the page with a timer to indicate the user is in focus mode. The navigation bar should have also disappeared, and the tester should expect to be unable to access any other page in the app.
- The focus timer and Pomodoro timers should display the correct seconds and minutes.
- The buttons on the Pomodoro timer should work as expected, and the timer should visually change when Pause and Skip are clicked.
- The Statistics page should show the correct focus time and number of Pomodoros the user has completed, and the leader-board should visually display the rankings as expected.

Date Last Performed:

This test was last performed on Saturday 28th March 2026

Pass or Fail:

When tested, this test passed, and the final version of the code works exactly as intended.

Chapter 5

Project management

5.1 Management

The project was managed using a combination of Git for version control and Trello to track task ownership and progress across the team. Git proved invaluable in allowing us to work on separate features simultaneously without conflict, and the shared Trello board gave everyone visibility into what was being worked on at any given time. This structure kept the project organised and ensured accountability, as each team member could see how their contributions fit into the wider picture.

5.2 What Worked Well

A key part of our workflow was to hold regular biweekly meetings, which we made a point of conducting in person whenever possible. Face-to-face interaction consistently proved more effective than remote alternatives, as being physically present with one another made discussions feel more engaging and led to more productive conversations around upcoming tasks. The informal dynamic of in-person meetings also made it easier to raise concerns early, helping us avoid issues that compound over time. We also developed a clearer understand of the individual strengths of each team member early on, with different people gravitating naturally towards different areas, whether frontend development, backend logic, or documentation, and we leaned into this by aligning task assignments accordingly.

5.3 What Didn't Work Well

There were, of course, challenges along the way. Coordination occasionally broke down when workloads became uneven, and there were moments where communication gaps led to duplicated effort or misaligned expectations. For many members, working within a group of this size was a new experience entirely, and learning how to navigate shared responsibilities across a larger team took time to adjust to. We overcame this together by leaning more heavily into Git as a collaborative tool and by placing a greater emphasis on open communication, both of which helped us build a more structured and reliable way of working as the project matured. We addressed the broader coordination challenges by being more deliberate in our meetings about explicitly assigning ownership of tasks and setting clearer deadlines, which improved our overall cohesion in the latter stages of the project.

5.4 What The Team Learned

On a broader level, the project was a significant learning experience both technically and interpersonally. As software engineers, we each grew more comfortable working within

a shared codebase, and making design decisions under time pressure. Perhaps more importantly, we came to appreciate the human side of software development. Effective collaboration requires more than technical skill; it demands clear communication, a willingness to adapt, and mutual trust in your teammates. Leaving the project, we carried with us not only a stronger technical foundation but also a more grounded understanding of what it means to work as part of a cohesive engineering team.